

Hypothesise and Enhance

django-helpdesk

Team: Team 4
Members: Manoj. C, Meghana. S, Aseem

Date: Day 9 (Feature Enhancement Activity)
Repository: github.com/MANOFHATERS/DAY2-APPLIED-PROGRAMMING

Our task

We were asked to choose at least two enhancements from Appendix A, turn each customer need into a clear behavioural requirement using our model of the existing system, and then implement and validate the change. This worksheet records our work for both enhancements in the same format as the worked example discussed in class. We deliberately kept the language plain and our reasoning visible, so that another person can follow us from the existing UI to the final validated change.

We chose to do the work UI-first. We ran the demo application, signed in as each role (customer, support agent, manager), opened the screens we cared about and wrote down what we could see and do. Only after we had those observations did we look at the code to confirm our hypotheses and to implement the change. We did this because our teacher was clear that going straight to the code is not what is wanted for this activity.

Our two selected enhancements:

- Enhancement 1 — A.2 Waiting for Customer: a new ticket status so agents can mark a ticket that is blocked waiting for information from the customer.
- Enhancement 2 — A.1 Transfer a ticket to another queue: a dedicated Transfer action on the ticket detail page so moving a ticket between queues is no longer buried inside the general Edit form.

Our model evolves as the work proceeds: M0 is the model before today's changes, M1 is the model after Enhancement 1 has been understood, implemented and validated, and M2 is the model after Enhancement 2 has also been completed. The model changes for each enhancement are described inside the worksheets below.

Enhancement 1 Worksheet

Selected Enhancement 1: A.2 Waiting for Customer

0.1 Elaborate the customer need into a requirement

Customer need (Appendix A.2): Support agents sometimes cannot continue working on a ticket until the customer provides additional information. Today there is no clear ticket state that distinguishes this situation from work that the support team can actively continue. We would like agents to be able to mark a ticket as Waiting for Customer, and later resume work when the required information has been received.

Before deciding anything we walked through the running application as the agent Anita and observed what the ticket screens actually let us do.

What we observed in the running UI:

- On the ticket detail page there is a Respond tab. The status of the ticket is shown as a set of radio buttons. For an Open ticket the visible options were Open, Resolved, Closed, Duplicate.
- For a ticket that is already Resolved, the radio options changed to Reopened, Resolved, Closed. We did not see a Waiting option in either list.
- The dashboard shows tickets grouped by status, and the ticket list filters by status. Both use the same five labels (Open, Reopened, Resolved, Closed, Duplicate).
- There is a Hold button in the Actions menu, but that is about the support team pausing work for their own reasons (an internal pause), not about waiting for the customer to come back with information.

From these observations, an agent who is blocked on the customer has no honest way to make that visible. They either leave the ticket as Open, which makes the ticket look like active work to anyone scanning the dashboard, or close it, which loses the fact that the team still intends to work on it. We treated that as the behaviour the customer need is asking us to change.

Changes we propose to X, U, and/or f:

Let X be the LTS state of a ticket. Before today we modelled the status component of X as the finite set {Open, Reopened, Resolved, Closed, Duplicate}. We add one new status value to X:

X adds: Waiting for Customer

Let U be the set of actions an agent can perform on a ticket. We add no brand new action — moving a ticket into and out of Waiting is done through the existing ticket-update action. What changes is the set of status transitions f that this update action is allowed to perform. Concretely:

f adds: (Open, Waiting) via update (set status = Waiting)

f adds: (Reopened, Waiting) via update (set status = Waiting)

f adds: (Waiting, Open) via update (resume work)

f adds: (Waiting, Resolved) via update (customer reply closes)

f adds: (Waiting, Closed) via update

f adds: (Waiting, Duplicate) via update

Transitions into Waiting are allowed only from Open or Reopened. We did not allow Resolved to Waiting because a ticket that is already Resolved is not active work, and we did not allow Closed to Waiting for the same reason. If a Closed ticket later turns out to need customer input, the existing Reopen transition brings it back to Reopened first, and from there it can move to Waiting.

Important CBIF components/interactions affected:

- Ticket component gains a new value for its status field. The Ticket component is otherwise unchanged.

- FollowUp component: each status change into or out of Waiting is recorded as a FollowUp entry, exactly the same way every other status change is recorded today. We did not have to add a new follow-up type — the existing one carries a new_status field and a comment, which is enough.
- Dashboard / Ticket list presentation components: these read the ticket status and group or filter by it. Because our new status sits in the same status field as the existing ones, the existing queries pick it up automatically once the status list is extended.

Behavioural transition table:

Current state	Action	Resulting state	Other effects / constraints
Open	update (set Waiting)	Waiting for Customer	FollowUp recorded; queue, owner, priority preserved
Reopened	update (set Waiting)	Waiting for Customer	FollowUp recorded; queue, owner, priority preserved
Waiting for Customer	update (set Open)	Open	Resume work. FollowUp recorded.
Waiting for Customer	update (set Resolved, Closed, or Duplicate)	Resolved, Closed, or Duplicate	Allowed if the customer reply makes one of these the right outcome
Resolved	update (set Waiting)	(rejected)	Not in flow. Use Reopen first if needed.

Our detailed requirement:

An agent viewing a ticket that is currently Open or Reopened must be able to set its status to Waiting for Customer from the same Respond tab used for any other status change. The ticket then displays the new status label and badge colour in the dashboard, the ticket list and the ticket detail page. While a ticket is Waiting for Customer, the agent must be able to set the status back to Open (resume work), or to move it forward to Resolved, Closed or Duplicate if the customer reply makes one of those the right outcome. Queue, owner and priority must be preserved across the Waiting transition. A ticket that is Resolved or Closed must not be allowed to go directly to Waiting — it must be Reopened first. Existing behaviour that must remain unchanged: the meaning of all five original statuses, the Hold / Unhold action, the Edit Ticket form, and the way FollowUp entries are recorded for status changes.

0.2 Hypothesise about how the relevant existing behaviour works

Before reading the relevant code in detail we proposed three explanations for how the ticket status radio buttons are produced on the ticket detail page. The evidence in this section is from the running UI and from reading model fields, not from pasting source code into the worksheet.

Hypothesis	Evidence for / against
H1 The set of status radio buttons shown for a ticket is computed from a single, fixed list of statuses that is shared by the whole application.	For: On every ticket screen we opened, the status options always came from the same small set of labels (Open, Reopened, Resolved, Closed, Duplicate). The labels were identical on the dashboard, the ticket list, and the ticket detail page. That suggested a single shared source of the list rather than per-screen hard-coding.
H2 The radio buttons shown for a ticket depend on the ticket's current status, via a per-status flow table that decides which destination statuses are valid.	For: When we viewed ticket 1 (Open) the radio buttons were Open, Resolved, Closed, Duplicate. When we viewed ticket 2 (Resolved) the radio buttons changed to Reopened, Resolved, Closed. The two sets differ, so the buttons cannot be a single fixed list — the current status influences the choices offered. This became our strongest explanation.

Hypothesis	Evidence for / against
H3 The status radio buttons are produced by the form's status field directly from the model's choices attribute, with no per-status filtering.	Against: If the radio buttons were the full model choices list they would be the same set on every ticket. They are not (see H2), so this explanation does not fit the evidence.

Our final explanation of how the relevant existing behaviour works:

The ticket detail page asks the ticket for the list of status choices that are valid from its current status, and renders that list as radio buttons. The current status therefore acts as the source of the per-status flow, and adding a new status with appropriate flow entries is enough to make that status appear in the right places in the UI. We did not have to change the template, the dashboard or the ticket list — they all read the same shared status list.

0.3 Hypothesise about the change

We considered two possible places to introduce the Waiting status.

Possible change	Advantages / problems
F1 Extend the shared status list (choices, open-status list, status flow table, status CSS class map) with the new Waiting status. No template or view changes.	Closest to the existing pattern. The radio buttons are produced by reading the shared list, so adding the new status to that list automatically puts it in the right place on every screen. Risk: the rest of the codebase must already be reading the shared list and not a hard-coded copy of it. We checked this against the running UI before deciding.
F2 Add a separate Mark as Waiting button on the ticket detail page with its own view, form and follow-up writing, leaving the existing status radio buttons alone.	Very visible in the UI. Problems: it introduces a second, parallel path for changing status, which conflicts with the way every other status change is performed (via the existing update form). It also requires new template, view and URL work, and risks the dashboard and ticket list forgetting to count the new state because they read the standard status list.

Which approach will you implement, and why?

We chose F1. The existing system already has a single shared place that defines the status list, the per-status flow, the open-status list and the status badge colour. Adding the new status in that one place keeps the cause (the status definition) and the effect (the radio buttons, the dashboard count, the badge colour) visibly connected, and it does not introduce a second way to change status. F2 would have been more visible in the short term but would have left the codebase with two parallel mechanisms for the same thing, which we did not want.

Change hypothesis:

If our explanation is correct, adding the Waiting status to the shared status list and adding the right entries to the per-status flow table will make the new status appear as a radio button on Open and Reopened tickets, will keep it off Resolved and Closed tickets, will make the dashboard and ticket list recognise the new state, and will not change the behaviour of any of the five existing statuses.

0.4 Design tests before implementing

Test scenario	Expected result
T1 Open ticket 1, view the Respond tab.	The radio button set includes Open, Resolved, Closed, Duplicate, Waiting for Customer.
T2 Set ticket 1 to Waiting for Customer via the update form.	Ticket 1 status becomes Waiting for Customer. The ticket detail page, the ticket list and the dashboard show the new status. A FollowUp entry is recorded.

Test scenario	Expected result
T3 From the Waiting ticket, set status back to Open.	Ticket 1 status becomes Open. The transition is recorded as a FollowUp. Queue, owner and priority are unchanged from before T2.
T4 View ticket 2 (Resolved) and inspect the radio buttons.	The radio button set is Reopened, Resolved, Closed. Waiting for Customer does not appear — the flow restriction works.
T5 Edit ticket 1's title only, leaving status on Open.	Status remains Open; the new rule is not triggered by unrelated updates. Regression check that the new status did not break ordinary edits.

0.5 Implement, validate, and refine the model

Final result and anything we had to revise:

All five scenarios passed on the first attempt after we added the new status to the four shared places (choices, open-status list, per-status flow table, and CSS class map). We did not have to revise the change. T1 showed the new radio button alongside the existing four. T2 successfully moved the ticket to Waiting and recorded the FollowUp entry. T3 returned the ticket to Open with the queue, owner and priority preserved. T4 confirmed that a Resolved ticket does not show the Waiting option, because the flow table does not include the Waiting status in the list of valid destinations from Resolved. T5 confirmed that an unrelated edit leaves status alone.

We also ran the project's existing test suite (ticket actions, ticket views, navigation, status CSS classes, dashboard sorting). All previously passing tests still pass, which gave us evidence that the existing behaviour was not broken by the new status.

Important code locations investigated and what we learned:

- The shared helpdesk status settings module: this is where the existing status constants, choices list, open-status list, per-status flow table, and status CSS class map are defined. Reading it confirmed H2 - the radio buttons come from the flow table.
- The ticket detail template: this template renders the radio buttons by iterating over the per-status flow list exposed by the ticket. Reading it confirmed that adding the new status to the flow table is enough to make the new button appear, with no template change.
- The ticket model: this is where the status field and its `get_allowed_status_flow` property live. Reading it confirmed that the per-status flow is the source the UI uses, and that no other code path recomputes the status list independently.

How M0 changed to become M1:

M0 modelled ticket status as the set {Open, Reopened, Resolved, Closed, Duplicate} with transitions defined by a per-status flow table. M1 adds the new status Waiting for Customer to that set, adds the transitions Open to Waiting and Reopened to Waiting, and adds the resumption transitions Waiting to Open / Resolved / Closed / Duplicate. The LTS action set U is unchanged — the existing ticket-update action performs the new transitions. The CBIF component boundaries are also unchanged; only the Ticket component's status value set and the status flow inside it changed.

Git tag for this point: enhancement-1-complete

Enhancement 2 Worksheet

Enhancement 2 starts from the system after Enhancement 1. We use M1 as the starting model and the code tagged enhancement-1-complete as the starting implementation.

Selected Enhancement 2: A.1 Transfer a ticket to another queue

0.6 Elaborate the customer need into a requirement

Customer need (Appendix A.1): Support teams sometimes need to hand a ticket from one queue to another because another team is better placed to handle it. Today, changing the queue is treated as part of editing a ticket. We would like Transfer to be a clear ticket action of its own. A transfer should move the ticket to the selected destination queue and should work sensibly with the destination queue's default owner and the ticket's existing owner. The transfer should also be visible in the ticket's history.

As in Enhancement 1, we walked through the running UI before deciding anything.

What we observed in the running UI:

- On the ticket detail page, the Actions dropdown (top-right of the ticket header) contains only Edit, Hold / Unhold, and Delete. There is no Transfer entry.
- The only way we could find to move a ticket to a different queue was to click Edit, which opens a large form containing Title, Queue, Submitter E-Mail, Description, Priority, Due on, Secret key, KB item, Merged to. The Queue dropdown is the third field on that form, sandwiched between Title and Submitter E-Mail.
- When we submitted the Edit form after changing the queue, the queue did change, but the FollowUp history showed only a generic queue-change entry. It did not look like a deliberate transfer; it looked like one of several edits that happened to include a queue change.
- The ticket detail page also has a small Queue field in the description card, but it is read-only there — it just shows the current queue as text. So the agent has to leave the ticket detail page and go to the Edit form to move the ticket between queues.

Changes we propose to X, U, and/or f:

X is unchanged. Queue is already part of the ticket state in M1, so we do not need to add a new state component. U gains a new action, and f gains the transitions performed by that action.

U adds: `Transfer(ticket_id, destination_queue, comment)`

f adds: `(queue=q, ...) --Transfer(-->q') (queue=q', ...)`

The Transfer action takes a destination queue and an optional comment. The transition leaves all other ticket state (status, priority, owner, due date, etc.) unchanged. The destination queue is restricted to the queues the current user is allowed to access and excludes the ticket's current queue, so the action cannot be a no-op.

Interaction with the destination queue's default owner: if the destination queue has a default owner and the ticket is currently unassigned, the ticket's owner is set to that default owner as part of the transfer. If the ticket is already assigned, we leave the owner alone. We chose this so that the destination team is notified through assignment when the ticket has no owner, while not silently re-assigning a ticket that someone is already actively working on.

Important CBIF components/interactions affected:

- Ticket component: queue field is changed; owner field may be changed as described above. No new field is added.
- Queue component: read-only role here — the destination queue provides its default owner if needed. The Queue component itself is unchanged.

- FollowUp component: a new follow-up entry is created with the title "Ticket transferred from <src> to <dst>", the acting user, the current time, the optional comment, and a public flag. A TicketChange entry is added under that follow-up recording the old and new queue values. This is the same pattern the existing Hold / Unhold and status-change actions already use, so the new transfer is visible in history in the same shape as the rest of the ticket's history.
- Interaction with Enhancement 1: a Waiting-for-Customer ticket can be transferred too. The transfer leaves the ticket in the Waiting status, and the receiving team will see it as Waiting in their queue. We deliberately did not change the status during transfer.

Behavioural transition table:

Current state	Action	Resulting state	Other effects / constraints
Ticket with queue=q	Transfer(destination=q', comment)	Ticket with queue=q'	FollowUp "transferred from q to q'" recorded; status and priority unchanged; owner set to default owner of q' if ticket was unassigned
Ticket with queue=q	Transfer(destination=q)	(rejected)	Same-queue transfer is not allowed; the form is shown again with an error
Ticket with queue=q	Transfer(destination=q', comment)	Ticket with queue=q'	If q' has no default owner or ticket already has an owner, owner is preserved

Our detailed requirement:

On the ticket detail page, the Actions dropdown must contain a new Transfer entry that opens a dedicated transfer page. The transfer page shows the ticket's current queue, a dropdown of valid destination queues (the queues the agent can access, minus the current queue), an optional comment field, and a Transfer ticket submit button. On submit, the ticket's queue is updated to the chosen destination, a FollowUp entry titled "Ticket transferred from <src> to <dst>" is created with the agent as user and the optional comment as follow-up comment, and a TicketChange row recording the old and new queue is stored under that follow-up. If the destination queue has a default owner and the ticket was previously unassigned, the ticket is assigned to that default owner; otherwise the existing owner is preserved. Status and priority are never changed by Transfer. Submitting the current queue is rejected and the form is shown again with an error. Existing behaviour that must remain unchanged: the existing Edit form still allows changing the queue alongside other fields (we did not remove that); the existing FollowUp, TicketChange and Hold / Unhold behaviour; and all Enhancement 1 behaviour, including the Waiting for Customer status.

0.7 Hypothesise about how the relevant existing behaviour works

We proposed three explanations for how the queue is currently changed.

Hypothesis	Evidence for / against
H1 The queue can only be changed via the Edit Ticket form, which saves the entire ticket with the new queue as one of many fields, and produces a single generic follow-up for the whole edit.	For: In the running UI the only way we could find to change the queue was via the Edit form, and the only history entry produced was a generic edit follow-up. We did not see a dedicated Transfer button on the ticket detail page or anywhere else. This became our strongest explanation.
H2 The queue can also be changed through the small ticket-update form on the ticket detail page (the one used for status and owner changes), and that path produces a queue-change follow-up.	Against: The small update form on the ticket detail page does have a hidden queue field, but it is hidden precisely so that queue changes go through the Edit form. The form's UI does not expose queue as an editable field, so a user cannot pick a new queue from that path.

Hypothesis	Evidence for / against
H3 There is already a Transfer action somewhere in the application that we missed, and the customer need is asking us to expose it.	Against: We opened every link in the navigation sidebar (Dashboard, All Tickets, Saved Queries, New Ticket, Reports, Knowledgebase) and every entry in the Actions dropdown on the ticket detail page. None of them was a Transfer action. So if such an action exists it is not reachable from the UI, which is effectively the same as not existing for the customer.

Our final explanation of how the relevant existing behaviour works:

Today the queue is changed only as part of editing a ticket. The Edit Ticket form treats the queue as one of several editable fields and saves the whole ticket; the history records the change as a generic queue change inside an edit. There is no dedicated Transfer action in the UI and no dedicated follow-up title for transfers. This is exactly what the customer need describes as the behaviour to be changed.

0.8 Hypothesise about the change

We considered two possible implementations for the Transfer action.

Possible change	Advantages / problems
F1 Add a dedicated Transfer view and URL, a small Transfer template with just a destination-queue dropdown and an optional comment, and a Transfer link in the Actions dropdown. The view writes a labelled FollowUp and a TicketChange row.	Closest to the existing pattern (the Hold / Unhold action is implemented the same way: a dedicated view, a small interaction, a labelled follow-up). The Transfer action is then visible in the UI and visible in history. Risk: a new view is a new code path — we have to make sure it does not bypass the queue-change rules that the Edit form already enforces.
F2 Add a Transfer button that simply navigates the user to the existing Edit form with the queue field pre-highlighted, and change the Edit form so that, when only the queue has changed, it writes a labelled transfer follow-up instead of a generic edit one.	Reuses the existing form. Problems: the action is still inside the Edit form, which is exactly what the customer need says is wrong. The heuristic "only the queue has changed" is fragile — a queue change with a small description edit would be mislabelled. We rejected this approach.

Which approach will you implement, and why?

We chose F1. The customer need explicitly says Transfer should be a clear action of its own, not part of editing a ticket. F1 makes Transfer its own action in the Actions menu, its own page, its own view, and its own labelled follow-up in history. F2 keeps Transfer inside the Edit form, which contradicts the customer need. F1 also matches the way the existing Hold action is implemented, so it fits the existing patterns in the codebase.

Change hypothesis:

If our explanation is correct, adding a dedicated Transfer view, a small Transfer template, a Transfer link in the Actions dropdown, and a labelled follow-up will make Transfer visible as its own action on the ticket detail page, will move the ticket to the selected destination queue in a single step, will record the transfer as a labelled FollowUp entry visible in history, will leave status and priority unchanged, and will not affect the existing Edit Ticket form's ability to change the queue alongside other fields.

0.9 Design tests before implementing

These tests cover Enhancement 2 and also include regression tests for Enhancement 1.

Test scenario	Expected result
T1 Open ticket 1, open the Actions dropdown.	The dropdown contains Edit, Transfer, Hold, Delete. The new Transfer entry is visible as its own action.

Test scenario	Expected result
T2 Click Transfer, choose People Operations as the destination, write a comment, and submit.	Ticket 1 queue becomes People Operations. Status remains Open. Priority remains 3 (Normal). A FollowUp titled "Ticket transferred from IT Support to People Operations" appears in history with the comment.
T3 From the People Operations ticket, click Transfer again, choose IT Support, submit.	Ticket 1 queue becomes IT Support again. A second labelled transfer FollowUp appears in history. The TicketChange rows record the queue moving back.
T4 Click Transfer on ticket 1 (now in IT Support) and submit without choosing a destination, or choose IT Support itself.	The form is shown again with an error message. Ticket 1 queue remains IT Support. Same-queue transfer is rejected.
T5 Mark ticket 1 as Waiting for Customer (Enhancement 1), then Transfer it to People Operations.	Ticket 1 is in People Operations queue and still has the Waiting for Customer status. Both enhancements coexist: the transfer did not reset the Waiting state. Regression: Enhancement 1 status survives a transfer.

0.10 Implement, validate, and refine the model

Final result and anything we had to revise:

All five scenarios passed. T1 showed the new Transfer entry in the Actions dropdown. T2 transferred ticket 1 from IT Support to People Operations and recorded the labelled follow-up with the comment, leaving status and priority unchanged. T3 transferred the ticket back to IT Support and recorded a second labelled follow-up. T4 rejected the same-queue transfer and re-displayed the form with an error, leaving the queue unchanged. T5 confirmed that a Waiting-for-Customer ticket can be transferred without losing its Waiting status — the two enhancements coexist.

One small revision: our first version of the Transfer view did not explicitly exclude the ticket's current queue from the list of valid destinations on the POST path — it only excluded it from the dropdown shown on the GET page. We noticed this while writing T4 and added the same check on the POST path so that a crafted request cannot transfer a ticket to its own queue. After the revision, T4 passed.

We also ran the project's existing test suite and added a small set of new tests for both enhancements. The existing tests still pass and the new tests pass, which gave us evidence that both the new behaviour and the existing behaviour are correct.

Important code locations investigated and what we learned:

- The ticket detail template and its included description card template: this is where the Actions dropdown is rendered. Reading it confirmed H1 — the dropdown contained only Edit, Hold / Unhold and Delete, with no Transfer.
- The existing Hold / Unhold view in the staff views module: this was the closest pattern to what we needed. It is a small dedicated view that performs one action, writes a labelled FollowUp, writes a TicketChange row, and redirects back to the ticket. We used the same pattern for Transfer.
- The existing Edit Ticket view and form: this confirmed that the queue is one of many fields on the general edit form, which is what the customer need asks us to stop relying on. We deliberately did not change the Edit form — the existing way to change the queue alongside other fields still works.
- The URL configuration: this is where we added the new transfer URL pattern, next to the existing edit and hold patterns, so the new action is routed in the same way as the existing ones.

How M1 changed to become M2:

M1 already had Ticket with a queue field and FollowUp with a labelled title and an optional comment. M2 adds the Transfer action to U with the transition described in section 0.6. No new state component is added to X. The CBIF component boundaries are unchanged; only the Ticket component gains a new way to have its queue field changed, and the FollowUp component gains a new labelled title it can carry. The interaction with

Enhancement 1 is captured in M2 by noting that Transfer does not change status, so a Waiting ticket stays Waiting across a transfer.

Git tag for this point: enhancement-2-complete

Final Reflection

1. What did you learn while doing Enhancement 1 that made Enhancement 2 easier, or changed how you approached it?

Enhancement 1 taught us that the existing system already had a single shared place that defined the ticket status list, the per-status flow, and the badge colours, and that the UI read from that shared place wherever it needed to show status. That made us start Enhancement 2 by looking for the same kind of shared place for ticket actions. We found that the Actions dropdown on the ticket detail page was the equivalent shared place for actions, and the existing Hold action was the closest template for what we needed. Because we had already learned the pattern from Enhancement 1, we wrote Enhancement 2 to follow it (a small dedicated view, a labelled follow-up, a TicketChange row), rather than reaching for a more elaborate design that would have introduced a second pattern.

2. Did Enhancement 2 force you to revise anything you thought you understood after Enhancement 1? Explain.

Yes. After Enhancement 1 we had a working assumption that adding a new behaviour to this codebase was always a matter of extending a shared list and the UI would pick it up automatically. Enhancement 2 forced us to revise that: a new action on the ticket detail page is not produced from a shared list, it has to be added explicitly to the Actions dropdown, routed through a new URL, and rendered by a new view. We also had to revise our assumption that the same-queue check was only needed on the dropdown shown to the user — we added it to the POST handler after T4 made the gap visible. So Enhancement 2 changed our model of how actions reach the UI, and our model of where validation has to live.

3. Identify one part of M2 for which you now have stronger evidence than you had in M0. What is that evidence?

The CBIF component boundary between Ticket and FollowUp. In M0 we modelled FollowUp as a passive log of past events: a ticket changed, and a follow-up was written to record that the change happened. After both enhancements we have stronger evidence that FollowUp is the shared mechanism the codebase uses to make any state change visible in history — both the new status transitions (Enhancement 1) and the new Transfer action (Enhancement 2) write a labelled FollowUp and a TicketChange row, exactly the way the existing Hold / Unhold action does. The evidence is that we did not need to introduce any new history mechanism to make either enhancement visible in the ticket's history; we just used the existing FollowUp + TicketChange pattern with a new labelled title. That is now part of M2.

AI Use and Declaration

We used AI as a tool while completing this activity. The decisions about how to interpret the customer needs, which hypotheses to keep, which implementation approach to choose, which test scenarios to write, and how to revise the change after a test failed were made by our team. The important claims in this worksheet were checked against the running application, the project's existing test suite, and our own new tests.

We used AI to:

- help us locate where the ticket status list and per-status flow are defined in the codebase, after we had already observed the per-status behaviour in the UI.
- explain the existing Hold / Unhold view and template, which we then used as the pattern for the new Transfer view.
- suggest edge cases for our tests, including the same-queue transfer case (T4 for Enhancement 2).
- suggest two possible places where the new Waiting status could be implemented (our F1 and F2).

AI influenced our requirement decisions or implementation by:

suggesting that putting the new Waiting-for-Customer rule in the shared status list would propagate it everywhere through the UI automatically. We did use that suggestion, but only after checking against the running UI that the radio buttons really are produced from the shared list. AI also suggested an alternative in which the new status would be a separate button with its own view (our F2); we did not use that suggestion because, after walking through the UI and discussing the model, we concluded it would create a second parallel path for status changes and would not stay in sync with the dashboard and ticket list.

We checked these suggestions or claims by:

- walking through the running application as customer, agent and manager before writing anything down, and writing the observations from the UI into this worksheet as the evidence in the hypothesis tables.
- running the project's existing test suite after each enhancement and confirming that previously passing tests still pass.
- writing our own new tests for both enhancements and running them. The new tests live in `tests/test_team4_enhancements.py` in the repository.
- capturing screenshots of each test scenario running in the live UI; these are used as the evidence in the demo recording that accompanies this worksheet.

The important decisions our team made ourselves were:

- the choice of the two enhancements (A.2 Waiting for Customer and A.1 Transfer).
- the flow restriction that Resolved and Closed tickets cannot go directly to Waiting — this came out of our discussion, not from an AI suggestion.
- the rule that the destination queue's default owner is used only when the ticket is currently unassigned — we wrote this rule and the reasoning is in the Enhancement 2 worksheet.
- the decision to reject same-queue transfers on the POST path as well as the GET path, which we added after our own test T4 showed the gap.
- the final detailed requirements, the test scenarios, the implementation choices (F1 in both cases) and the explanations in the reflection.